Assignment 6: Logistic Regression

Copyright and Fair Use

This material, no matter whether in printed or electronic form, may be used for personal and non-commercial educational use only. Any reproduction of this material, no matter whether as a whole or in parts, no matter whether in printed or in electronic form, requires explicit prior acceptance of the authors.

Automatic Testing Guidelines

Automatic unittesting requires you to submit a notebook which contains strictly defined objects. Strictness of definition consists of unified shapes, dtypes, variable names and more.

Within the notebook, we provide detailed instruction which you should follow in order to maximise your final grade.

Name your notebook properly, follow the pattern in the template name:

Assignment_N_NameSurname_matrnumber

1. N - number of assignment
2. NameSurname - your full name where every part of the name starts with a capital letter, no spaces
3. matrnumber - you student number on ID card (with k, potentially with a leading zero)

Don't add any cells but use the ones provided by us. All cells have a unique ID so that the unit test can find it, so please do not add or remove any cell!

Always make sure that implemented functions have the correct output and given variables contain the correct data type. In the descriptions for every function you can find information on what datatype an output should have and you should stick to that in order to minimize conflicts with the unittest. Don't import any other packages than listed in the cell with the "imports" tag.

Questions are usually multiple choice (except the task description says otherwise) and can be answered by changing the given variables to either "True" or "False". "None" is counted as a wrong answer in any case!

Note: Never use variables you defined in another cell in your functions directly; always pass them to the function as a parameter. In the unitest, they won't be available either. If you want to make sure that everything is executable for the unittest, try executing cells/functions individually (instead of running the whole notebook).

In [1]:

```
# Nothing to do here, just run the cell.
import numpy as np
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

RSEED = 44
```

Task 1: Implementation of Logistic Regression

The goal of this exercise is to implement logistic regression from scratch using only numpy - do not import any new packages!

Complete the following tasks:

- **Code 1.1**:
  - Implement a function `calc_loss(w, X, y)`, that takes a parameter vector `w`, a data matrix `X` and a binary label vector `y` and returns the binary cross-entropy loss:

$$L_{BCE} = -\sum_{n=1}^{N} \boldsymbol{y}^n \ln(\sigma(\boldsymbol{x}^n \cdot w)) + (1 - \boldsymbol{y}^n) \ln(1 - \sigma(\boldsymbol{x}^n \cdot w))$$

- **Code 1.2**:
  - Implement the formula for the gradient computed in the lecture.
  - In particular you should implement a function `analytical_gradient(w, X, y)` that takes the same parameters as the function before but returns the gradient $\frac{\partial L}{\partial \mathbf{w}}$, where L is the binary cross-entropy loss.
- **Code 1.3**:
  - Test whether the gradient calculated by `analytical_gradient(w, X, y)` is correct via Gradient Checking. To do so, implement the function `numerical_gradient(w, X, y, loss_fn, eps)` that takes the same parameters as `logistic_gradient` as well as a loss function and an additional optional parameter `eps` and computes:

$$\text{numerical gradient} = \frac{f(x + \epsilon) - f(x - \epsilon)}{2\epsilon}$$

- **Code 1.4**:
  - Implement the function `generate_random(N, K)` that generates a random data matrix consisting of `N` data points with `K` features drawn from a standard normal distribution as well as corresponding random binary labels and a random weight vector, whose entries again stem from the standard normal distribution. Hint: to generate the distributions use `np.random.normal` and `np.random.randint`.
- **Question 1.5**:
  - Answer some questions about your results from the previous exercies.
- **Question 1.6**:
  - Answer some questions about the missing bias term.

Code 1.1 (5 Points):

In [2]:
```
def calc_loss(w: np.ndarray, X: np.ndarray, y: np.ndarray) -> float:
    """Function that computes the cross-entropy loss.
```

```
    Parameters
    ----------
    w : (K,) np.ndarray
        Weight vector.
    X : (N, K) np.ndarray
        Data matrix.
    y : (N,) np.ndarray
        Label vector.

    Returns
    -------
    float
        The cross-entropy loss.
    """
    # YOUR CODE HERE
    xnw = X @ w
    sigma_xnw = 1 / (1 + np.exp(-xnw))
    loss = -np.sum(y * np.log(sigma_xnw) + (1 - y) * np.log(1 - sigma_xnw
    return loss
```

In [3]:
```
# DO NOT DELETE THIS CELL!
N, K = 20, 5
w_testing = np.linspace(-3, 1, K)
X_testing = np.linspace(-2, 2, N*K).reshape(N, K)
y_testing = np.zeros(N)
y_testing[[2, 5, 7, 8, 9, 14]] = 1

loss = calc_loss(w_testing, X_testing, y_testing)
assert isinstance(loss, float), "The resulting loss is not a float!"
assert np.isclose(loss, 41.6671, atol=1e-4), "The loss is incorrect!"
```

Code 1.2 (5 Points):

In [4]:
```
def analytical_gradient(w: np.ndarray, X: np.ndarray, y: np.ndarray) -> n
    """Function that computes the logistic gradient via the cross-entropy

    Parameters
    ----------
    w : (K,) np.ndarray
        Weight vector.
    X : (N, K) np.ndarray
        Data matrix.
    y : (N,) np.ndarray
        Label vector.

    Returns
    -------
    gradient : (K,) np.ndarray
        Gradient vector.
    """
    # YOUR CODE HERE
    xnw = X @ w
    sigma_xnw = 1 / (1 + np.exp(-xnw))
    gradient = np.sum((sigma_xnw - y)[:, np.newaxis] * X, axis=0)
    return gradient
```

In [5]:
```
# DO NOT DELETE THIS CELL!
analytical_grad = analytical_gradient(w_testing, X_testing, y_testing)
```

```
assert isinstance(analytical_grad, np.ndarray), "The gradient is not a np
np.testing.assert_array_almost_equal(analytical_grad, np.array([-7.682233
```

Code 1.3 (14 Points):

In [6]:
```
def numerical_gradient(w: np.ndarray, X: np.ndarray, y: np.ndarray, loss_
    """Function that computes the numerical gradient for each epsilon in
    Hint: use the previously implemented cost function.

    Parameters
    ----------
    w : (K,) np.ndarray
        Weight vector.
    X : (N, K) np.ndarray
        Data matrix.
    y : (N,) np.ndarray
        Label vector.
    loss_fn: callable
        Loss function that calculates the cross-entropy loss.
    eps : float = 1e-7
        The epsilon value for the difference quotient calculation.

    Returns
    -------
    gradient : (K,) np.ndarray
        The numerical gradient approximating the analytical one.
    """
    # YOUR CODE HERE
    gradient = np.zeros_like(w)
    for i in range(len(w)):
        w_plus = w.copy()
        w_plus[i] += eps

        w_minus = w.copy()
        w_minus[i] -= eps
        gradient[i] = (loss_fn(w_plus, X, y) - loss_fn(w_minus, X, y)) /
    return gradient
```

In [7]:
```
# DO NOT DELETE THIS CELL!
numerical_grad = numerical_gradient(w_testing, X_testing, y_testing, lamb

assert isinstance(numerical_grad, np.ndarray), "The numerical gradient is
np.testing.assert_array_almost_equal(numerical_grad, np.array([0.00986604
```

Code 1.4 (10 Points):

In [8]:
```
def generate_random(N: int, K: int) -> tuple[np.ndarray, np.ndarray, np.n
    """Function that generates a random data matrix X and the random bina

    Important: To generate the distributions use numpy's np.random.<...>

    Parameters
    ----------
    N : int,
        Number of samples to generate.
    K : int,
        number of features of each sample.
```

```
    Returns
    -------
    tuple of np.ndarray
        - X_random : (N, K) np.ndarray
            The generated data matrix.
        - y_random : (N,) np.ndarray
            The generated binary labels vector.
        - w_random : (K,) np.ndarray
            The generated weights.
    """
    # YOUR CODE HERE
    X_random = np.random.normal(0, 1, (N, K))
    y_random = np.random.randint(0, 2, N)
    w_random = np.random.normal(0, 1, K)

    return X_random, y_random, w_random
```

In [9]:
```
# DO NOT DELETE THIS CELL!
np.random.seed(RSEED)
X_random, y_random, w_random = generate_random(5, 10)

assert isinstance(X_random, np.ndarray) and isinstance(y_random, np.ndarr
assert X_random.shape == (5, 10) and y_random.shape == (5,) and w_random.
assert np.all(np.isin(y_random, [0, 1])), "The generated labels vector ha
```

In [10]:
```
# Nothing to do here, just run the cell.
np.random.seed(RSEED)
N = 5
K = 10
eps_list = [1e-1, 1e-4, 1e-11]
X_random, y_random, w_random = generate_random(N, K)
analytical_grad = analytical_gradient(w_random, X_random, y_random)
numerical_grads = []
for eps in eps_list:
    numerical_grads.append(numerical_gradient(w_random, X_random, y_rando

comparison_results=[np.allclose(analytical_grad, numerical_grad, atol=1e-
# Check outputs
assert len(comparison_results) == len(eps_list), "List with comparison re
assert isinstance(comparison_results[0],bool), f"Comparison results shoul
print("Your randomly generated data:\n")
print("X =",X_random,"\n")
print("y =",y_random,"\n")
print("w = ",w_random,"\n")
print("Logistic gradient:\n",analytical_grad,"\n")
for i, res in enumerate(comparison_results):
    eps_ = eps_list[i]
    print(70*"=")
    print(f"Numerical gradient {i} with epsilon = {eps_}:\n", numerical_g
    print("Vectors within absolute tolerance of 10^-7: ", res, "\n")
```

```
Your randomly generated data:

X = [[-0.75061472  1.31635732  1.24614003 -1.60491574 -1.46814368 -1.71507
046
   1.85878369  0.08758798 -0.0523222   0.55547164]
 [-0.96340369 -0.18032147 -1.18340916  0.60544592 -0.95165055  0.36085606
   1.06061026 -0.11715218  0.82566485 -1.20981355]
 [-1.19050362  0.21987182 -0.21291913 -1.41149914 -0.48298102  1.20176208
  -0.70580303  0.02518135 -0.39112815  0.19313912]
 [ 0.34929093  0.59118838  0.52040987  0.82522239  0.42620351  0.19564003
  -0.51411218 -3.07368882 -0.39515907 -0.79539918]
 [-0.70320151 -1.73739268  0.75080826 -1.86395314  0.41851098 -0.23533384
   0.43210717  0.87029193  2.00862219  1.37177091]]

y = [1 1 0 1 0]

w =  [-0.78250832 -0.68105798  0.76098988  0.62483292 -0.45678902 -0.33331
661
  0.55805765  1.39253236  0.44025472 -1.24948056]

Logistic gradient:
 [-0.99891843 -2.25379004 -0.13244644 -2.38498447  0.17958543  0.23018978
  0.26027996  3.72772198  1.95281213  1.91449719]

======================================================================
Numerical gradient 0 with epsilon = 0.1:
 [-0.99900268 -2.25334646 -0.13270225 -2.38377877  0.18012599  0.23127324
  0.2590955   3.72660128  1.95152582  1.91418285]

Vectors within absolute tolerance of 10^-7:  False

======================================================================
Numerical gradient 1 with epsilon = 0.0001:
 [-0.99891843 -2.25379004 -0.13244644 -2.38498447  0.17958543  0.23018978
  0.26027996  3.72772198  1.95281213  1.91449719]

Vectors within absolute tolerance of 10^-7:  True

======================================================================
Numerical gradient 2 with epsilon = 1e-11:
 [-0.99897868 -2.25384156 -0.13238299 -2.38493669  0.17958968  0.23021585
  0.26019187  3.72772924  1.9528823   1.91446858]

Vectors within absolute tolerance of 10^-7:  False
```

Question 1.5 (3 Points):

Q: Following the results from your comparison function, if at all, why do you think is the right choice of ϵ crucial, for computing adequate approximations with the numerical gradient function?

a1_) The choice does not matter, as long as epsilon is in (0,1) the function will give a satisfying solution, according to our tolerance criterion.
b1_) Very large values for ϵ, e.g. $\epsilon = 0.1$, lead to a divergence of the gradient as the denominator approaches zero.
c1_) Very small values for ϵ, e.g. $\epsilon \leq 1\text{e-}11$, can result in calculation errors, this could

be due to issues with numerical precision, as both values in the numerator as well as in the denominator are very small.

To answer the question, assign `True` or `False` boolean values to variables in the next cell. For example, if you think that **a1_)** is correct, define a variable `a1_` and set it to `True`, the same applies to **b1_)** and the other options. A non-correctly answered question as well as no answer (i.e. answer "None") yields 0 points for a specific question.

In [11]:
```
# YOUR CODE HERE
#raise NotImplementedError()
a1_ = False
b1_ = False
c1_ = True
```

In [12]:
```
# DO NOT DELETE THIS CELL!
assert a1_ is not None, "Store True/False!"
assert a1_ in [True, False], "Invalid Answer!"
```

In [13]:
```
# DO NOT DELETE THIS CELL!
assert b1_ is not None, "Store True/False!"
assert b1_ in [True, False], "Invalid Answer!"
```

In [14]:
```
# DO NOT DELETE THIS CELL!
assert c1_ is not None, "Store True/False!"
assert c1_ in [True, False], "Invalid Answer!"
```

Question 1.6 (3 Points):

Q: Did you notice that we left out the bias term in our implementation of the logistic regression? (In the lecture Unit 5, Slide 45/47 shows the logistic unit with bias.) The bias b represents the log-odds of the probability that the dependent variable takes on the value of 1 when all independent variables are set to zero.

Why was it okay to not use a bias for our data?

d1_) The bias term cancels out when calculating the derivative of the loss-function, because it does not depend on $\mathbf{X}$.
e1_) The bias is only relevant when all independent variables are exactly 0, which never actually happens in real data.
f1_) The data follows a standard normal distribution, hence a bias of 0 leads to the correct predictions, based on the condition that the label distribution is symmetric.

To answer the question, assign `True` or `False` boolean values to variables in the next cell. For example, if you think that **d1_)** is correct, define a variable `d1_` and set it to `True`, the same applies to **e1_)** and the other options. A non-correctly answered question as well as no answer (i.e. answer "None") yields 0 points for a specific question.

In [15]:
```
# YOUR CODE HERE
d1_ = False
```

```
e1_ = False
f1_ = True
```

In [16]:
```
# DO NOT DELETE THIS CELL!
assert d1_ is not None, "Store True/False!"
assert d1_ in [True, False], "Invalid Answer!"
```

In [17]:
```
# DO NOT DELETE THIS CELL!
assert e1_ is not None, "Store True/False!"
assert e1_ in [True, False], "Invalid Answer!"
```

In [18]:
```
# DO NOT DELETE THIS CELL!
assert f1_ is not None, "Store True/False!"
assert f1_ in [True, False], "Invalid Answer!"
```

Task 2: Logistic Regression on a real dataset

Next we intend to apply logistic regression on a real data set. Firstly, in the following cell the data set `DataSet_LR_a.csv` is loaded and split into a training set and a test set ($50\,\%$ each). Also a constant feature (all ones) is added to X such that we can train an additional weight for the intercept which is the same for all datapoints. We visualize the dataset and then you should do the following:

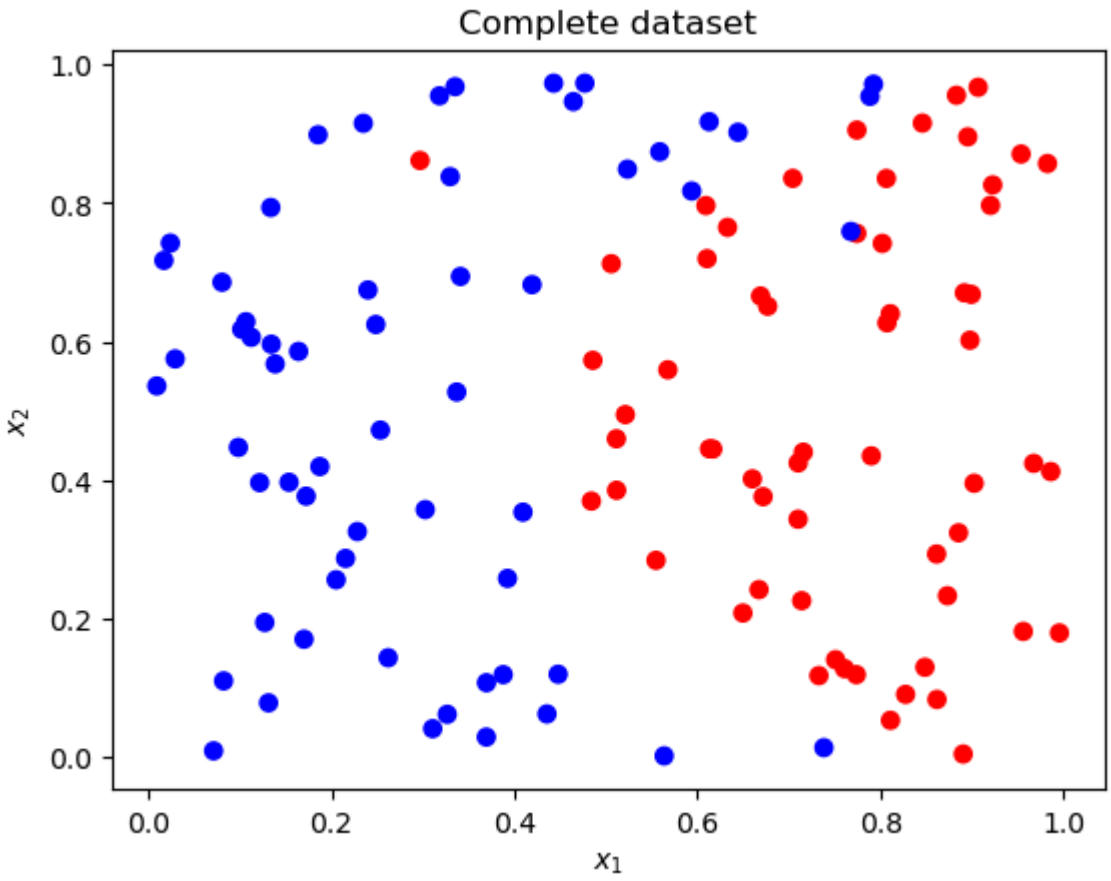
- **Code 2.1**:
  - Implement a function `fit_Log_Reg_Model(...)` that uses Logistic Regression with Gradient Descent to train classifiers on the training set.
  - Use randomly initialized weights, drawn from a uniform distribution between -1 and 1 (have a look at `np.random.unifrom`), a default learning rate `eta` and a maximum number of iterations `max_iter`.
  - Furthermore the algorithm should stop either if the difference between the loss of the last iteration step and the current loss is less than the `stopping_criterion` or the maximum of allowed iterations `max_iter` is reached (in this case print that the maximum iteration was reached).
  - Also print the losses in 10000 step intervals (also at step 0!). The function should return the fitted model weights.
- **Code 2.2**:
  - Furthermore, implement a function `predict_Log_Reg(w, X_test)` that returns the prediction for the given parameter vector $\mathbf{w}$ and data matrix $\mathbf{X}_\mathbf{t}\mathbf{est}$.
- **Question 2.3**:
  - Answer some questions about your results.
- **Code 2.4**:
  - Implement the function `calc_metrics(prediction, targets, threshold)` that computes the Accuracy and Balanced Accuracy. To do so, you should classify samples as class `1` if the Logistic Regression returns values $\geq$ `threshold` and `0` otherwise.
- **Plot 2.5**:

- Plot the ROC curve of the classifiers on the test samples and compute and display the corresponding AUC. Hint: the functions `roc_curve` and `auc` from `sklearn.metrics` might be useful.

In [19]:
```
# Nothing to do here, just run the cell.
# Read data, split into X(features) and y(labels)
Z = np.genfromtxt('DataSet_LR_a.csv', delimiter=',',skip_header=1)
X, y = Z[:,:-1], Z[:,-1]
# Prepend ones for intercept
X = np.hstack((np.ones((X.shape[0],1)),X))

# Plot data distribution
color= ['red' if elem==1 else 'blue' for elem in y ]
plt.scatter(X[:,-2], X[:,-1], c=color)
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.title('Complete dataset')

# Split into test and training set
X_train=X[:int(X.shape[0]/2)]
X_test=X[int(X.shape[0]/2):]
y_train=y[:int(len(y)/2)]
y_test=y[int(len(y)/2):]
```



Code 2.1 (20 Points)

In [20]:
```
def fit_Log_Reg_Model(
    X_train: np.ndarray,
    y_train: np.ndarray,
    loss_fn: callable,
```

```
    grad_fn: callable,
    eta: float = 1e-4,
    stopping_criterion: float = 1e-5,
    max_iter: float = 1e5
) -> np.ndarray:
    """Function that fits a logistic regression model to the given data.
    Prints the loss every 10000 steps (also in the beginning at step 0).
    Training stops if either the change of the current loss to the loss o

    Important: Make sure to initialize the weights-vector from a random u

    Parameters
    ----------
    X_train : (N, K) np.ndarray
        Data matrix.
    y_train : (N,) np.ndarray
        Label vector.
    loss_fn: callable
        Loss function that calculates the binary cross-entropy loss.
    grad_fn: callable
        Function that calculates the analytical gradient.
    eta : float
        Learning rate for gradient descent.
    stopping_criterion : float
        Stopping criterion for the training process.
    max_iter : int, optional
        Max iteration at which training should stop, by default 100000.

    Returns
    -------
    weights : np.ndarray
        The fitted weights.
    """
    # YOUR CODE HERE
    #print( X_train.shape)
    w = np.random.uniform(-1, 1, X_train.shape[1])
    loss = loss_fn(w, X_train, y_train)
    print(f"loss at step 0: {loss}")
    for i in range(int(max_iter)):
        w -= eta * grad_fn(w, X_train, y_train)
        new_loss = loss_fn(w, X_train, y_train)
        if i % 10000 == 0:
            print(f"loss at step {i + 1}: {new_loss}")
        if abs(new_loss - loss) < stopping_criterion or i == max_iter:
            break
        loss = new_loss
    return w
```

In [21]:

```
# DO NOT DELETE THIS CELL!
np.random.seed(RSEED)

w_learned = fit_Log_Reg_Model(X_train, y_train, calc_loss, analytical_gra
print("The learnt weights are: w =", w_learned, "\n")
assert isinstance(w_learned, np.ndarray), "The resulting weights are not
assert w_learned.shape == (X_train.shape[1],), "The resulting weights hav
np.testing.assert_almost_equal(w_learned, np.array([-3.26516093,  8.25477
```

```
loss at step 0: 47.65980541875169
loss at step 1: 47.65437838310059
loss at step 10001: 25.564029244046267
loss at step 20001: 21.28621318781567
loss at step 30001: 19.760173812462448
loss at step 40001: 19.049649228929283
loss at step 50001: 18.669119400981526
loss at step 60001: 18.44713344927724
loss at step 70001: 18.310026448086543
The learnt weights are: w = [-3.26516093  8.25477584 -1.49905643]
```

Code 2.2 (5 Points)

In [22]:
```
def predict_Log_Reg(w: np.ndarray, X_test: np.ndarray) -> np.ndarray:
    """Function that calculates the prediction for one or more new and un

    Do not use any packages for this task except numpy.

    Parameters
    ----------
    w : (K,) np.ndarray
        Vector of trained weights.
    X_test : (N, K) np.ndarray
        Samples to make predictions for.

    Returns
    -------
    prediction : (N,) np.ndarray
        Vector of calculated predictions, floats between 0 and 1
    """
    # YOUR CODE HERE
    xnw = X_test @ w
    prediction = 1 / (1 + np.exp(-xnw))
    return prediction
```

In [23]:
```
# DO NOT DELETE THIS CELL!
pred = predict_Log_Reg(np.linspace(-5, 5, X_train.shape[1]), X_train[:5])
assert isinstance(pred, np.ndarray), "The predictions are not a np.ndarra
assert np.all(pred >=0) and np.all(pred <= 1), "The predictions should be
np.testing.assert_array_almost_equal(pred, np.array([0.05192384, 0.006805
```

Now we fit the logistic regression model to the training data with different learning rates and print the parameters for the test data. In case you want, you can try out different learning rates (just change the `eta_list`).

In [24]:
```
# Fit the model on the training data with different learning rates.
pred_train = []
pred_test = []
eta_list = [1e-7, 1e-4, 6e-1]

for lr in eta_list:
    np.random.seed(RSEED)
    w_learned = fit_Log_Reg_Model(X_train, y_train, calc_loss, analytical
    pred_train.append(predict_Log_Reg(w_learned, X_train))
    pred_test.append(predict_Log_Reg(w_learned, X_test))
    print("The learnt weights are: w =", w_learned, "\n")
```

```
loss at step 0: 47.65980541875169
loss at step 1: 47.65979999130486
The learnt weights are: w = [ 0.66968399 -0.79040721  0.48928063]

loss at step 0: 47.65980541875169
loss at step 1: 47.65437838310059
loss at step 10001: 25.564029244046267
loss at step 20001: 21.28621318781567
loss at step 30001: 19.760173812462448
loss at step 40001: 19.049649228929283
loss at step 50001: 18.669119400981526
loss at step 60001: 18.44713344927724
loss at step 70001: 18.310026448086543
The learnt weights are: w = [-3.26516093  8.25477584 -1.49905643]

loss at step 0: 47.65980541875169
loss at step 1: 31.125543514129134
loss at step 10001: 30.192030537344777
loss at step 20001: 32.10252248304022
loss at step 30001: 42.50338063494557
loss at step 40001: 35.79685913979725
loss at step 50001: 30.4280153058447
loss at step 60001: 35.72319112211846
loss at step 70001: 53.11108715304457
loss at step 80001: 30.156895478978445
loss at step 90001: 32.56893329276275
The learnt weights are: w = [-6.17209681 24.80854336 -1.16601192]
```

Let's visualize your results!

In [25]:

```
# Nothing to do here, just run the cell.
# Plot training and test dataset as well as the predictions for both
for i, _ in enumerate(pred_train):
    fig = plt.figure(figsize = (12,10))
    fig.suptitle(f"Results for eta = {eta_list[i]}")
    plt.subplot(2, 2, 1)
    color= ['red' if elem>0.5 else 'blue' for elem in y_train ]
    plt.scatter(X_train[:,-2], X_train[:,-1], c=color,label='the data')
    plt.xlabel('$x_1$')
    plt.ylabel('$x_2$')
    plt.title('Training dataset')

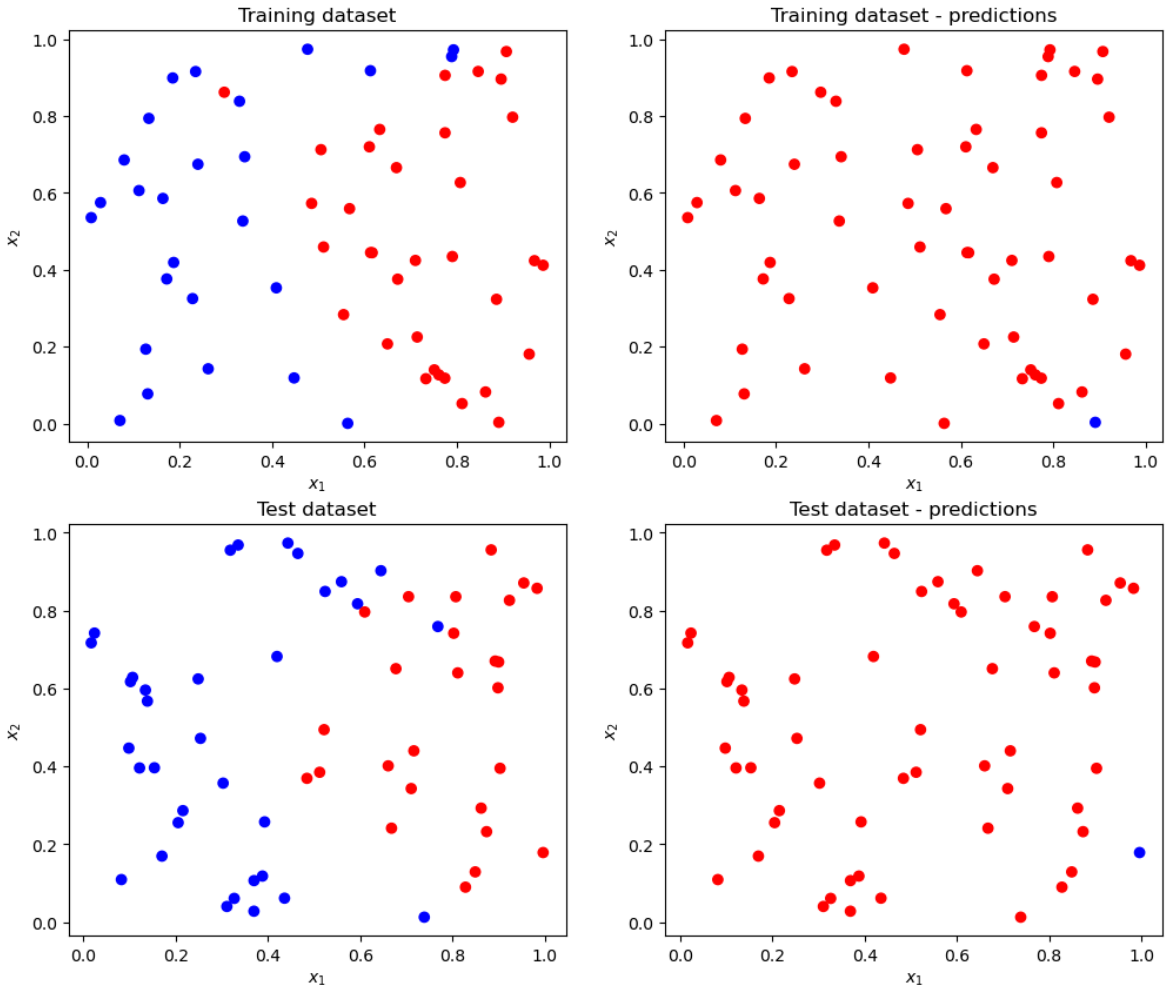
    plt.subplot(2, 2, 2)
    color= ['red' if elem>0.5 else 'blue' for elem in pred_train[i] ]
    plt.scatter(X_train[:,-2], X_train[:,-1], c=color,label='the data')
    plt.xlabel('$x_1$')
    plt.ylabel('$x_2$')
    plt.title('Training dataset - predictions')

    plt.subplot(2, 2, 3)
    color= ['red' if elem>0.5 else 'blue' for elem in y_test ]
    plt.scatter(X_test[:,-2], X_test[:,-1], c=color, label='the data')
    plt.xlabel('$x_1$')
    plt.ylabel('$x_2$')
    plt.title('Test dataset')

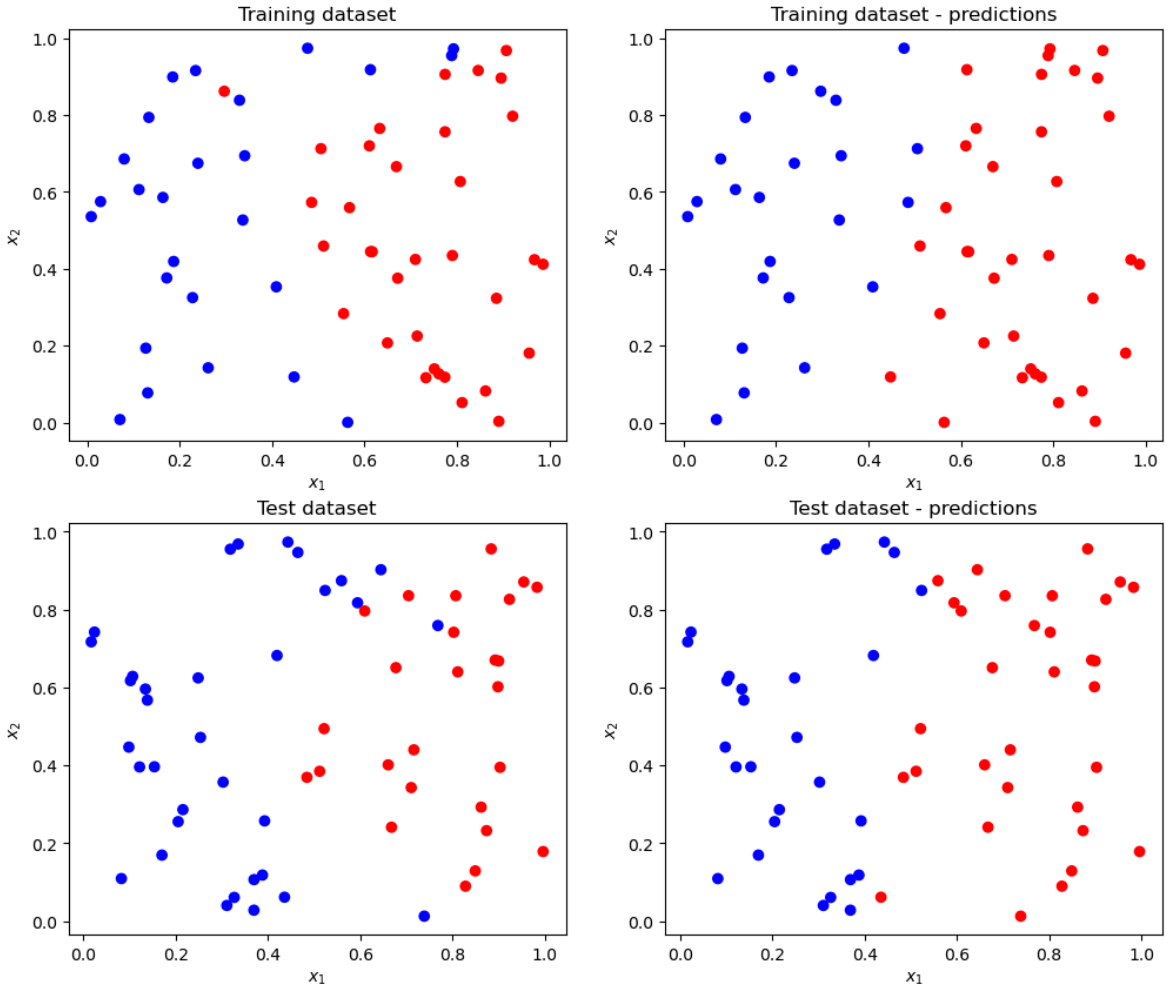
    plt.subplot(2, 2, 4)
    color= ['red' if elem>0.5 else 'blue' for elem in pred_test[i] ]
    plt.scatter(X_test[:,-2], X_test[:,-1], c=color, label='the data')
```

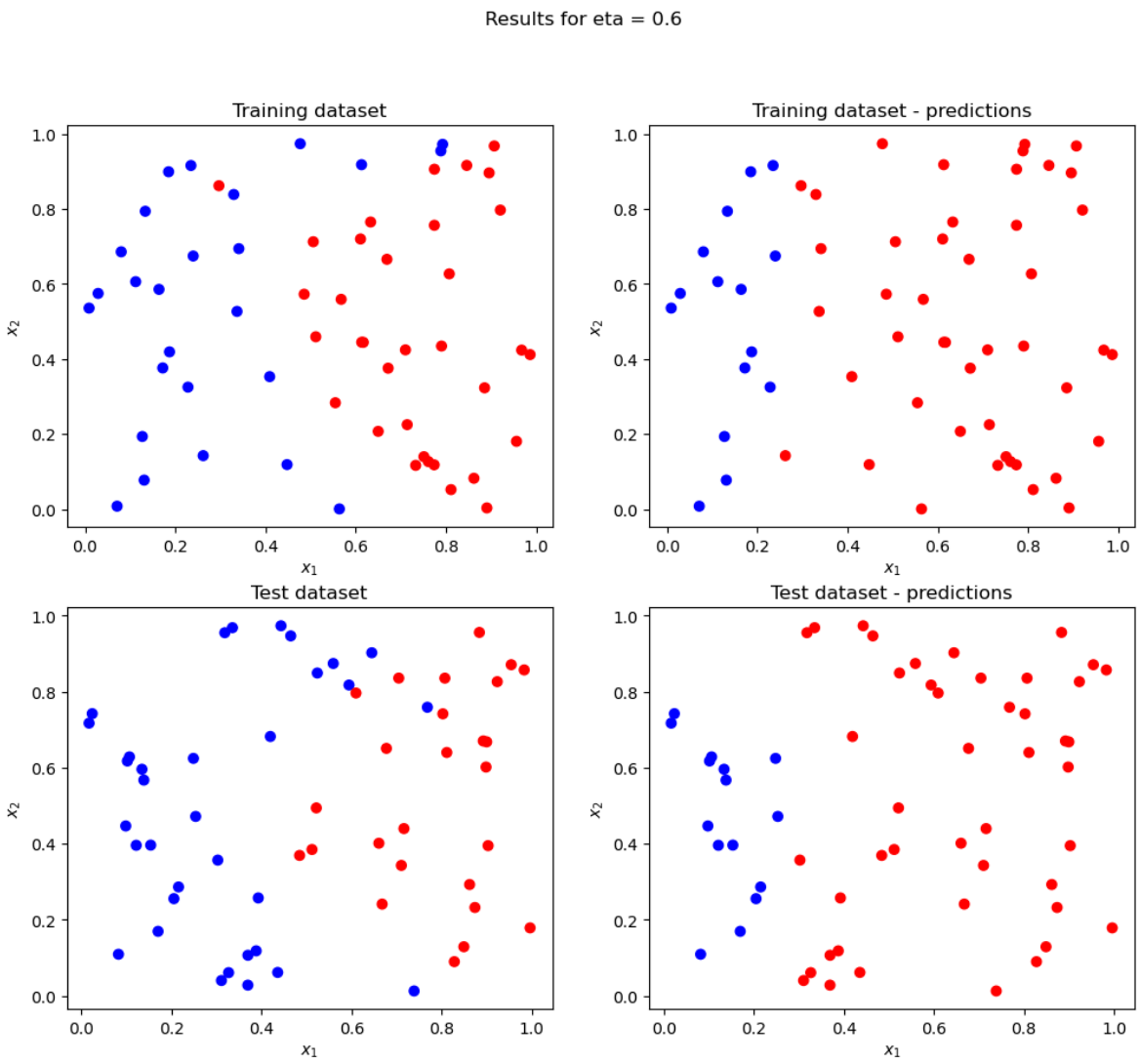
```
    plt.xlabel('$x_1$')
    plt.ylabel('$x_2$')
    plt.title('Test dataset - predictions')
```

Results for eta = 1e-07



Results for eta = 0.0001





Question 2.3 (5 Points):

Q: Following your experiments with the the learning rate (eta), which of the following statements are correct?

a2_) Learning rates > 0.5 lead to fast convergence (i.e. our default `stopping_criterion` was met) and reasonably good predictions on the test data.
b2_) For learning rates smaller than 1e-6 the algorithm does not stop with the default `stopping_criterion` before `max_iter` is reached.
c2_) When inspecting the plot for the default learning rate 1e-4, we see that also the outliers are classified correctly.
d2_) No matter what learning rate we choose from (0,1), if the algorithm converges, in the sense that it stops with the `stopping_criterion`, the resulting weights are always the same (within a small tolerance of $\sim |0.1|$)
e2_) If we trained the classifier for a very long time, with the optimal learning rate, the resulting model would separate the training data perfectly (accuracy = 1), but it would perform much worse on the test data (overfitting).

To answer the question, assign `True` or `False` boolean values to variables in the next cell. For example, if you think that **a2_)** is correct, define a variable `a2_` and set it to `True`, the same applies to **b2_)** and the other options. A non-correctly answered

question as well as no answer (i.e. answer “None”) yields 0 points for a specific question.

In [ ]:
```
# YOUR CODE HERE
#raise NotImplementedError()
a2_ = True
b2_ = False
c2_ = False
d2_ = False
e2_ = True
```

In [27]:
```
# DO NOT DELETE THIS CELL!
assert a2_ is not None, "Store True/False!"
assert a2_ in [True, False], "Invalid Answer!"
```

In [28]:
```
# DO NOT DELETE THIS CELL!
assert b2_ is not None, "Store True/False!"
assert b2_ in [True, False], "Invalid Answer!"
```

In [29]:
```
# DO NOT DELETE THIS CELL!
assert c2_ is not None, "Store True/False!"
assert c2_ in [True, False], "Invalid Answer!"
```

In [30]:
```
# DO NOT DELETE THIS CELL!
assert d2_ is not None, "Store True/False!"
assert d2_ in [True, False], "Invalid Answer!"
```

In [31]:
```
# DO NOT DELETE THIS CELL!
assert e2_ is not None, "Store True/False!"
assert e2_ in [True, False], "Invalid Answer!"
```

Code 2.4 (15 Points)

In [58]:
```
def calc_metrics(predictions: np.ndarray, targets: np.ndarray, threshold:
    """Function that calculates the accuracy as well as the balanced accu

    Again, only us numpy for this task.

    Parameters
    ----------
    predictions : np.ndarray
        The computed predictions between 0 and 1.
    targets : np.ndarray
        The true binary targets.
    threshold : float, optional
        Threshold for the label decision, by default 0.5.

    Returns
    -------
    tuple of float
        acc : float
            The calculated accuracy.
        balanced_acc :  float
            The calculated balanced accuracy.
    """
    # YOUR CODE HERE
```

```
    acc = np.mean((predictions >= threshold) == targets)
    tp = np.sum((predictions >= threshold) & (targets == 1))
    fn = np.sum((predictions < threshold) & (targets == 1))
    tn = np.sum((predictions < threshold) & (targets == 0))
    fp = np.sum((predictions >= threshold) & (targets == 0))
    balanced_acc = 0.5 * ((tp / (tp+fn)) + (tn / (tn+fp)))
    return acc, balanced_acc
```

In [60]:
```
# DO NOT DELETE THIS CELL!
acc, bacc = calc_metrics(np.linspace(0, 1, len(y_train)), y_train)
assert isinstance(acc, float), "The accuracy is not a float!"
assert np.isclose(acc, 0.5667, atol=1e-4), "The accuracy is not correct!"
```

In [61]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(bacc, float), "The balanced accuracy is not a float!"
assert np.isclose(bacc, 0.5679, atol=1e-4), "The balanced accuracy is not
```

In [62]:
```
# Nothing to do here, just run the cell.
# Calculate metrics for both the train and test set
np.random.seed(RSEED)
w_learned = fit_Log_Reg_Model(X_train, y_train, calc_loss, analytical_gra
pred_train = predict_Log_Reg(w_learned, X_train)
pred_test = predict_Log_Reg(w_learned, X_test)

result_train = calc_metrics(pred_train, y_train)
result_test = calc_metrics(pred_test, y_test)
print("\nAccuracy on training data:", result_train[-2])
print("Balanced accuracy on training data:", result_train[-1])
print("Accuracy on test data:", result_test[-2])
print("Balanced accuracy on test data:", result_test[-1])
```

```
loss at step 0: 47.65980541875169
loss at step 1: 47.65437838310059
loss at step 10001: 25.564029244046267
loss at step 20001: 21.28621318781567
loss at step 30001: 19.760173812462448
loss at step 40001: 19.049649228929283
loss at step 50001: 18.669119400981526
loss at step 60001: 18.44713344927724
loss at step 70001: 18.310026448086543

Accuracy on training data: 0.8666666666666667
Balanced accuracy on training data: 0.8597285067873304
Accuracy on test data: 0.9
Balanced accuracy on test data: 0.9117647058823529
```

Plot 2.5 (15 Points)

In [65]:
```
def roc_auc(y_test: np.ndarray, y_pred: np.ndarray):
    """Function that computes the AUC of the input predictions vs the tru

    You are only allowed to use the imported sklearn functions.

    Parameters
    ----------
    y_test : np.ndarray
        True labels of test data.
    y_pred : np.ndarray
```

```
        Predictions of test labels.
    """
    # YOUR CODE HERE
    fpr, tpr, thresholds = roc_curve(y_test, y_pred)
    roc_auc = auc(fpr, tpr)
    plt.figure()
    lw = 2
    label = f'area = {roc_auc}'
    plt.plot(fpr, tpr, color='red', lw=lw, label=label)
    plt.plot([0, 1], [0, 1], color='blue', lw=lw, linestyle='--')
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive')
    plt.ylabel('True Positive')
    plt.title('ROC AUC')
    plt.legend(loc="lower right")
    plt.show()
```

In [66]:
```
# Nothing to do here, just run the cell.
roc_auc(y_test, pred_test)
```

